Retrieval Augmented Generation (RAG) Workflow

Start coding or generate with AI.

STEP 1 — Install and Import Required Libraries

```
# Install necessary libraries. chromadb is a vector database and sentence-transformers is used for generating
%pip install chromadb sentence-transformers
```

```
Collecting chromadb
  Downloading chromadb-1.5.6-cp39-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (5.0 kB)
Requirement already satisfied: sentence-transformers in /usr/local/lib/python3.12/dist-packages (5.3.0)
Collecting build>=1.0.3 (from chromadb)
  Downloading build-1.4.2-py3-none-any.whl.metadata (5.8 kB)
Requirement already satisfied: pydantic>=2.0 in /usr/local/lib/python3.12/dist-packages (from chromadb) (2.12
Requirement already satisfied: pydantic-settings>=2.0 in /usr/local/lib/python3.12/dist-packages (from chromac
Collecting pybase64>=1.4.1 (from chromadb)
  Downloading pybase64-1.4.3-cp312-cp312-manylinux1_x86_64.manylinux2014_x86_64.manylinux_2_17_x86_64.manylinu
Requirement already satisfied: uvicorn>=0.18.3 in /usr/local/lib/python3.12/dist-packages (from uvicorn[standa
Requirement already satisfied: numpy>=1.22.5 in /usr/local/lib/python3.12/dist-packages (from chromadb) (2.0.2
Requirement already satisfied: typing-extensions>=4.5.0 in /usr/local/lib/python3.12/dist-packages (from chror
Collecting onnxruntime>=1.14.1 (from chromadb)
  Downloading onnxruntime-1.24.4-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (5.2 kB)
Requirement already satisfied: opentelemetry-api>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from chror
Collecting opentelemetry-exporter-otlp-proto-grpc>=1.2.0 (from chromadb)
  Downloading opentelemetry_exporter_otlp_proto_grpc-1.40.0-py3-none-any.whl.metadata (2.6 kB)
Requirement already satisfied: opentelemetry-sdk>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from chror
Requirement already satisfied: tokenizers>=0.13.2 in /usr/local/lib/python3.12/dist-packages (from chromadb)
Collecting pypika>=0.48.9 (from chromadb)
  Downloading pypika-0.51.1-py2.py3-none-any.whl.metadata (51 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 52.0/52.0 kB 2.7 MB/s eta 0:00:00
Requirement already satisfied: tqdm>=4.65.0 in /usr/local/lib/python3.12/dist-packages (from chromadb) (4.67.3
Requirement already satisfied: overrides>=7.3.1 in /usr/local/lib/python3.12/dist-packages (from chromadb) (7
Requirement already satisfied: importlib-resources in /usr/local/lib/python3.12/dist-packages (from chromadb)
```

```
Requirement already satisfied: grpcio>=1.58.0 in /usr/local/lib/python3.12/dist-packages (from chromadb) (1.86
Collecting bcrypt>=4.0.1 (from chromadb)
  Downloading bcrypt-5.0.0-cp39-abi3-manylinux_2_34_x86_64.whl.metadata (10 kB)
Requirement already satisfied: typer>=0.9.0 in /usr/local/lib/python3.12/dist-packages (from chromadb) (0.24.1
Collecting kubernetes>=28.1.0 (from chromadb)
  Downloading kubernetes-35.0.0-py2.py3-none-any.whl.metadata (1.7 kB)
Requirement already satisfied: tenacity>=8.2.3 in /usr/local/lib/python3.12/dist-packages (from chromadb) (9.1
Requirement already satisfied: pyyaml>=6.0.0 in /usr/local/lib/python3.12/dist-packages (from chromadb) (6.0.3
Requirement already satisfied: mmh3>=4.0.1 in /usr/local/lib/python3.12/dist-packages (from chromadb) (5.2.1)
Requirement already satisfied: orjson>=3.9.12 in /usr/local/lib/python3.12/dist-packages (from chromadb) (3.11
Requirement already satisfied: httpx>=0.27.0 in /usr/local/lib/python3.12/dist-packages (from chromadb) (0.28
Requirement already satisfied: rich>=10.11.0 in /usr/local/lib/python3.12/dist-packages (from chromadb) (13.9
Requirement already satisfied: jsonschema>=4.19.0 in /usr/local/lib/python3.12/dist-packages (from chromadb)
Requirement already satisfied: transformers<6.0.0,>=4.41.0 in /usr/local/lib/python3.12/dist-packages (from se
Requirement already satisfied: huggingface-hub>=0.20.0 in /usr/local/lib/python3.12/dist-packages (from senter
Requirement already satisfied: torch>=1.11.0 in /usr/local/lib/python3.12/dist-packages (from sentence-transfo
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (from sentence-transfor
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from sentence-transformers)
Requirement already satisfied: packaging>=24.0 in /usr/local/lib/python3.12/dist-packages (from build>=1.0.3-1
Collecting pyproject_hooks (from build>=1.0.3->chromadb)
  Downloading pyproject_hooks-1.2.0-py3-none-any.whl.metadata (1.3 kB)
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx>=0.27.0->chromadb)
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from httpx>=0.27.0->chroma
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx>=0.27.0->c
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-packages (from httpx>=0.27.0->chromadb)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->http
Requirement already satisfied: filelock>=3.10.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-h
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-h
Requirement already satisfied: hf-xet<2.0.0,>=1.4.2 in /usr/local/lib/python3.12/dist-packages (from huggingfa
Requirement already satisfied: attrs>=22.2.0 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=4.19
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: referencing>=0.28.4 in /usr/local/lib/python3.12/dist-packages (from jsonschema
```

```
import chromadb
from sentence_transformers import SentenceTransformer

print("Libraries installed and imported successfully.")
```

```
Libraries installed and imported successfully.
```

```
# Initialize the ChromaDB client. This sets up an in-memory client by default.
client = chromadb.Client()

# Create a new collection named "rag_collection". If a collection with this name already exists, it will be us
# A collection in ChromaDB is similar to a table in a relational database, where your data (embeddings, docume
collection = client.create_collection(name="rag_collection")

print(f"ChromaDB client initialized and collection '{collection.name}' created/accessed.")
```

```
ChromaDB client initialized and collection 'rag_collection' created/accessed.
```

```
# Load a pre-trained SentenceTransformer model. 'all-MiniLM-L6-v2' is a good
# This model will convert our text documents into numerical vector represent
model = SentenceTransformer('all-MiniLM-L6-v2')

print("Embedding model 'all-MiniLM-L6-v2' loaded successfully.")
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate li
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set
modules.json: 100%  349/349 [00:00<00:00, 16.6kB/s]

config_sentence_transformers.json: 100%  116/116 [00:00<00:00, 3.53kB/s]

README.md:  10.5k/? [00:00<00:00, 288kB/s]

sentence_bert_config.json: 100%  53.0/53.0 [00:00<00:00, 1.39kB/s]

config.json: 100%  612/612 [00:00<00:00, 21.5kB/s]

model.safetensors: 100%  90.9M/90.9M [00:02<00:00, 132MB/s]

Loading weights: 100%  103/103 [00:00<00:00, 259.80it/s, Materializing param=pooler.dense.weight]
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                     | Status     |  |
------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical a
tokenizer_config.json: 100%  350/350 [00:00<00:00, 10.0kB/s]

vocab.txt:  232k/? [00:00<00:00, 13.3MB/s]

tokenizer.json:  466k/? [00:00<00:00, 22.0MB/s]

special_tokens_map.json: 100%  112/112 [00:00<00:00, 9.95kB/s]

config.json: 100%  190/190 [00:00<00:00, 15.3kB/s]
Embedding model 'all-MiniLM-L6-v2' loaded successfully.
```

STEP 4 — Prepare Sample Documents

```
# Define a list of sample text documents that we want to store and query.
documents = [
    "Artificial Intelligence is the simulation of human intelligence.",
    "Machine Learning is a subset of AI.",
    "Deep Learning uses neural networks.",
    "Natural Language Processing deals with text data.",
    "RAG combines retrieval and generation."
]

# Assign unique IDs to each document. These IDs will be used to reference the documents in ChromaDB.
ids = [f"doc{i+1}" for i in range(len(documents))]

print(f"Prepared {len(documents)} sample documents with IDs: {ids}")
```

```
Prepared 5 sample documents with IDs: ['doc1', 'doc2', 'doc3', 'doc4', 'doc5']
```

STEP 5 — Convert Text to Embeddings

```
# Use the loaded SentenceTransformer model to convert the text documents into numerical embeddings.
# Each document will be transformed into a vector (a list of numbers).
embeddings = model.encode(documents)

print(f"Converted {len(documents)} documents into embeddings. Each embedding has a dimension of {embeddings.sh
# print(f"Example embedding for doc1:\n{embeddings[0][:5]}...") # Print first 5 elements for brevity
```

```
Converted 5 documents into embeddings. Each embedding has a dimension of 384.
```

STEP 6 — Store Embeddings in ChromaDB

```
# Add the documents, their corresponding embeddings, and unique IDs to the ChromaDB collection.
# This makes the data searchable within the vector database.
collection.add(
    documents=documents, # The original text content of the documents
    embeddings=embeddings, # The numerical vector representations of the documents
    ids=ids # Unique identifiers for each document
)

print(f"Added {len(documents)} documents and their embeddings to the collection '{collection.name}'.")
```

```
Added 5 documents and their embeddings to the collection 'rag_collection'.
```

STEP 7 — Perform Similarity Search

```
# Define a query string that we want to use to find similar documents.
query = "What is AI?"

# Convert the query string into an embedding using the same model used for the documents.
# The query also needs to be a vector to compare against the stored document vectors.
query_embedding = model.encode([query])

# Perform a similarity search in the ChromaDB collection.
# We are asking for the top 2 most similar results to our query embedding.
results = collection.query(
    query_embeddings=query_embedding.tolist(), # The embedding of our query (needs to be a list for ChromaDB)
    n_results=2, # Number of most similar results to retrieve
    include=['documents', 'distances'] # Specify what to include in the results (original documents and their
)

print(f"Query: '{query}'")
print("Search Results:")
print(results)
```

```
Query: 'What is AI?'
Search Results:
{'ids': [['doc1', 'doc2']], 'embeddings': None, 'documents': [['Artificial Intelligence is the simulation of hu
```

STEP 8 — Display Retrieved Results

```
# Iterate through the retrieved documents and print them.
# The results dictionary contains a list of lists for 'documents', so we access the first (and only) inner lis
print("Retrieved Documents:")
for i, doc in enumerate(results['documents'][0]):
    distance = results['distances'][0][i] # Get the distance for the current document
    print(f"- Document: '{doc}' (Distance: {distance:.4f})")
```

```
Retrieved Documents:
- Document: 'Artificial Intelligence is the simulation of human intelligence.' (Distance: 0.5604)
- Document: 'Machine Learning is a subset of AI.' (Distance: 0.7004)
```

STEP 9 — Build Simple Retrieval Function

```
def retrieve(query: str, n_results: int = 2) -> list[str]:
    """
    Retrieves semantically similar documents from the ChromaDB collection based on a given query.

    Args:
        query (str): The natural language query string.
        n_results (int): The number of top similar documents to retrieve. Defaults to 2.

    Returns:
        list[str]: A list of retrieved document strings.
    """
    # Encode the query into a vector embedding
    query_embedding = model.encode([query])

    # Perform the similarity search in the ChromaDB collection
    results = collection.query(
        query_embeddings=query_embedding.tolist(), # Convert numpy array to list for ChromaDB
        n_results=n_results, # Number of results to fetch
        include=['documents'] # We only need the document content
```

```
    )

    # Return the list of retrieved documents
    return results['documents'][0]

# Demonstrate the retrieval function
print("Demonstrating retrieval for 'Explain machine learning':")
retrieved_docs = retrieve("Explain machine learning")
for i, doc in enumerate(retrieved_docs):
    print(f"- {doc}")

print("\nDemonstrating retrieval for 'Neural networks explanation':")
retrieved_docs_nn = retrieve("Neural networks explanation")
for i, doc in enumerate(retrieved_docs_nn):
    print(f"- {doc}")
```

```
Demonstrating retrieval for 'Explain machine learning':
- Machine Learning is a subset of AI.
- Deep Learning uses neural networks.

Demonstrating retrieval for 'Neural networks explanation':
- Deep Learning uses neural networks.
- Machine Learning is a subset of AI.
```